



ABU-DHABI UNIVERSITY

College of Engineering & Computer Science

Department of Mechanical Engineering

COURSE NAME & NUMBER	MEC-465: NUMERICAL AND FE		
INSTRUCTOR	M. S. GADALA		
ASSIGNMENT OR PROJECT TITLE & NUMBER	Final course project		
DUE DATE	4/Decemeber/ 2017		
SUBMITTED BY: STUDENT NAME: LAST, FIRST STUDENT NUMBER, SIGNATURE	Maaz bin naseer	1051890	
	Muhammad Bilal Amir	1052776	

Table of Contents

List of Figures	1
Abstract	2
Introduction	2
Problem Statement	3
Matlab Code	4
Element Nodal Shape Function	4
Element Stiffness Matrix	5
Global Matrix	6
Results	7
1. MATLAB	7
2. Hand Calculation	8
Solution	8
3. ANSYS	12
Verifications of results	13
Conclusion	13
Bibliography	14
Appendix	14

List of Figures

Figure 1: Problem Description (Gadala, 2017)	3
Figure 2: Nodal Shape Function Script	4
Figure 3 Element Stiffness matrix symbolical	5
Figure 4 Global Matrix code	6
Figure 5 Global force vector	6
Figure 6 Global matrix result 1	7
Figure 7 Global matrix result 2	7
Figure 8 Reduced Matrix result	7
Figure 9: Sample problem bi-linear quadrilateral (Khennane, 2013)	8
Figure 10 Ansys Model	12
Figure 11 Stiffness matrix from Ansys	12

Abstract

The objective of this final mini-project was to familiarize ourselves with the understanding and development of MATLAB code to assess and derive the global stiffness matrix of a plane stress 4-node quadrilateral element. The output should be the shape function matrix, stiffness matrix and assembly of the matrix. Finally, the last part being conducting test cases and validating the code by using results from either Ansys or Workbench.

Introduction

Commercial FEM programs have been essential to the engineering field in making finite element analysis relative easy. By eliminating dependency on hand calculations and possibility of computing large matrices, it has made FEM programs more popular and widely employed. MATLAB program used in this project helps us with the finite element analysis of our selected model. Millions of engineers and scientific researches employ the use of MATLAB program to analyze and design systems and products. The matrix-based MATLAB language is the a widely used to express computational mathematics. Built-in graphical and plotting tools make it easy to visualize and gain insights from data. (MATLAB, 2017). This final mini-project is going to establish the idea that MATLAB can be used to conduct calculations on finite element problems and help solve a finite element problem and the results will be compared with that of commercial FEM programs like Ansys or Workbench.

Problem Statement

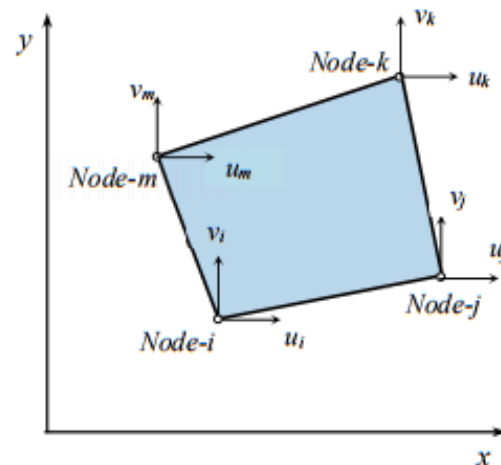
The figure shows a plane stress 4-node quadrilateral element with 2-DOF/node as shown. The stiffness matrix for this element may be written in the following general form (refer to your notes):

$$[K] = [k_{ij}], i, j = 1, 8$$

Also, given:

Young's modulus of the material, E ; Poisson's ν ;
 x, y coordinates of all nodes; thickness of the element, t .

It is required to:



- (a) Write a Matlab function to find the element nodal shape functions. The input to the function will be:
 - The coordinates (x and y -coordinates) of the three nodal points and their identification as nodes i, j, k, m .

The output of the function should be:

 - All the nodal shape functions: N_i, N_j, N_k and N_m .
- (b) Write a Matlab function to find the element stiffness matrix symbolically. The input to the function will be:
 - Nodes i, j, k, m and their coordinates as well as the element thickness, t .
 - Young's modulus of the material, E ; and Poisson's ν .

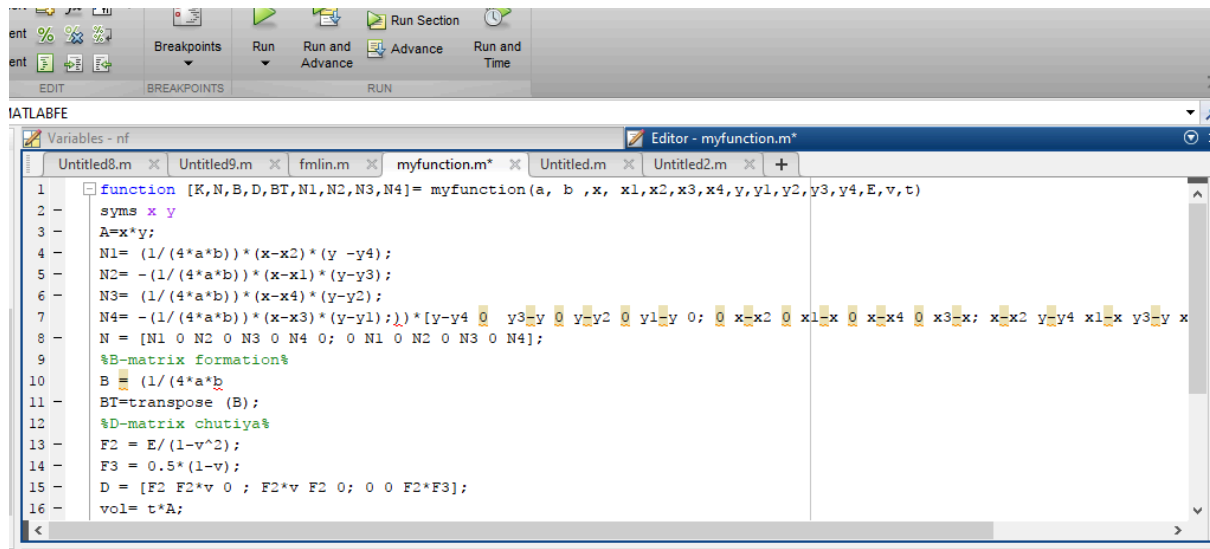
The output of the function should be:

 - All coefficients of the matrix k_{ij} , i and $j = 1, 8$.
- (c) Write a Matlab function to assemble the above matrix into a global matrix. The global matrix must be bigger size. The input to the function will be:
 - Nodes i, j, k, m .
 - All coefficients of the matrix k_{ij} , i and $j = 1, 8$.
 - The global matrix size $m \times m$, where $m > 8$.
- (d) Device an appropriate problem(s) or test cases to test your codes and functions. **Test cases must include five or more (better) elements.**
- (e) Validate your code and results by solving the test case(s) using Ansys or Workbench and comparing the results and giving enough comments.

Figure 1: Problem Description (Gadala, 2017)

Matlab Code

Element Nodal Shape Function

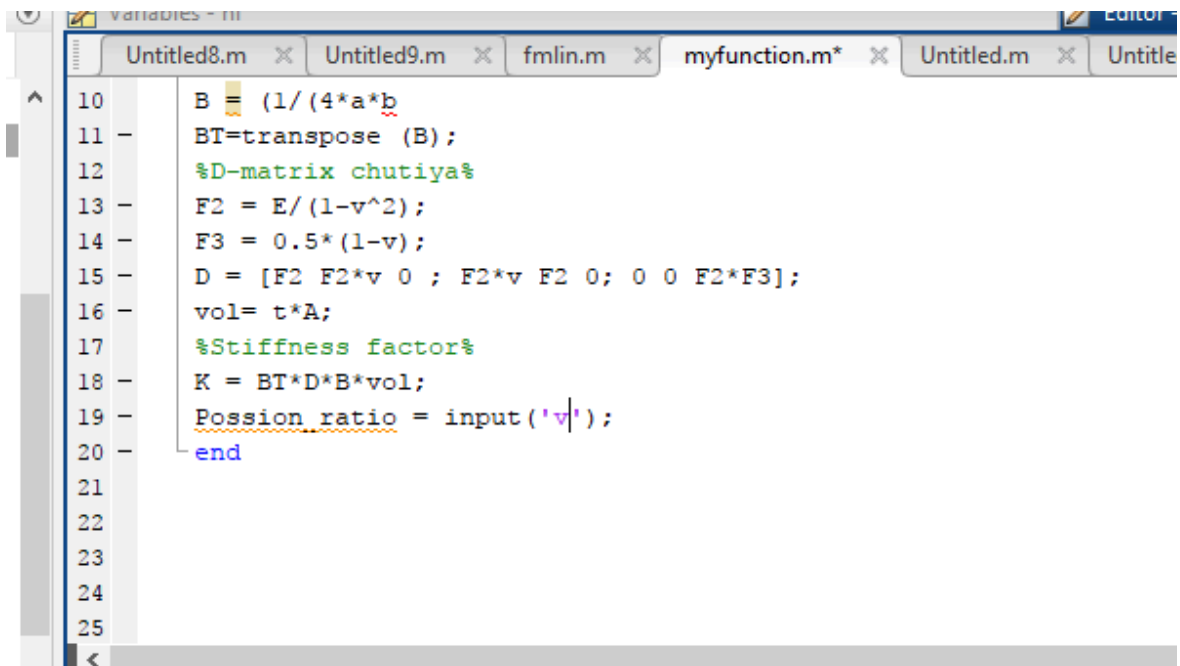


The image shows a MATLAB Editor window with the following script:

```
1 function [K,N,B,D,BT,N1,N2,N3,N4]= myfunction(a, b ,x, x1,x2,x3,x4,y,y1,y2,y3,y4,E,v,t)
2 -
3 - syms x y
4 - A=x*y;
5 - N1= (1/(4*a*b))*(x-x2)*(y -y4);
6 - N2= -(1/(4*a*b))*(x-x1)*(y-y3);
7 - N3= (1/(4*a*b))*(x-x4)*(y-y2);
8 - N4= -(1/(4*a*b))*(x-x3)*(y-y1);
9 - N = [N1 0 N2 0 N3 0 N4 0; 0 N1 0 N2 0 N3 0 N4];
10 - %B-matrix formation%
11 - B = (1/(4*a*b)
12 - BT=transpose (B);
13 - %D-matrix chutiya%
14 - F2 = E/(1-v^2);
15 - F3 = 0.5*(1-v);
16 - D = [F2 F2*v 0 ; F2*v F2 0; 0 0 F2*F3];
17 - vol= t*A;
```

Figure 2: Nodal Shape Function Script

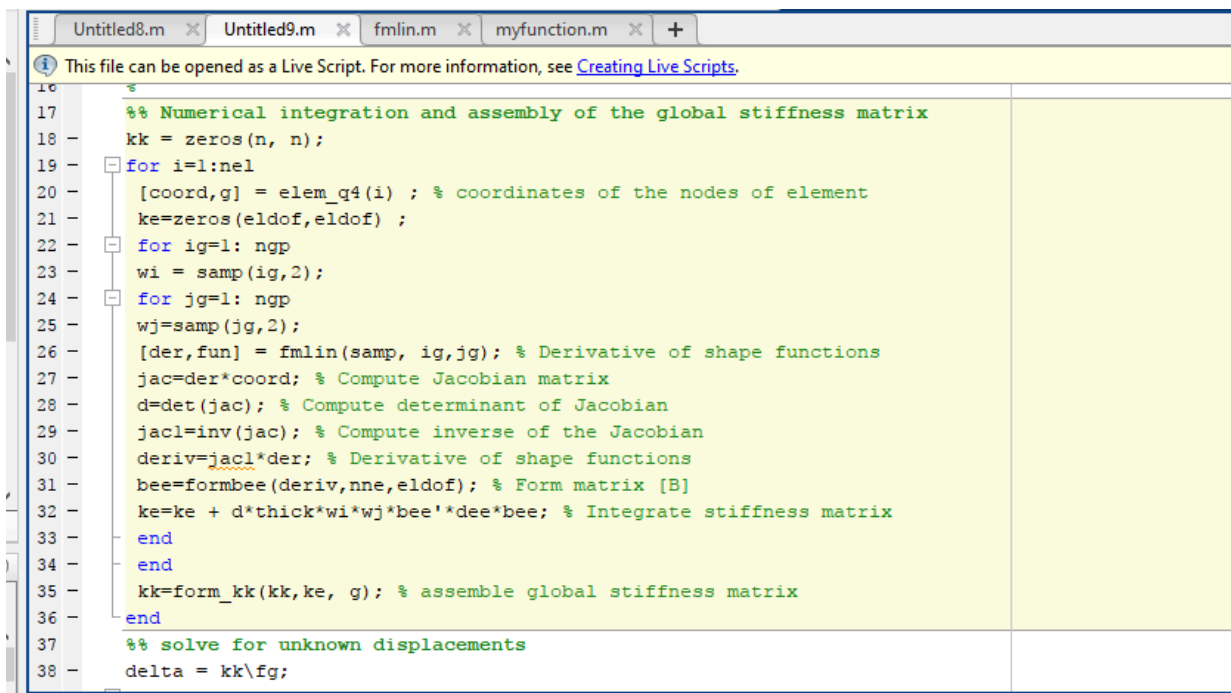
Element Stiffness Matrix



```
10 B = (1/(4*a*b  
11 BT=transpose (B);  
12 %D-matrix chutiya%  
13 F2 = E/(1-v^2);  
14 F3 = 0.5*(1-v);  
15 D = [F2 F2*v 0 ; F2*v F2 0; 0 0 F2*F3];  
16 vol= t*A;  
17 %Stiffness factor%  
18 K = BT*D*B*vol;  
19 Possion_ratio = input('v|');  
20 end  
21  
22  
23  
24  
25
```

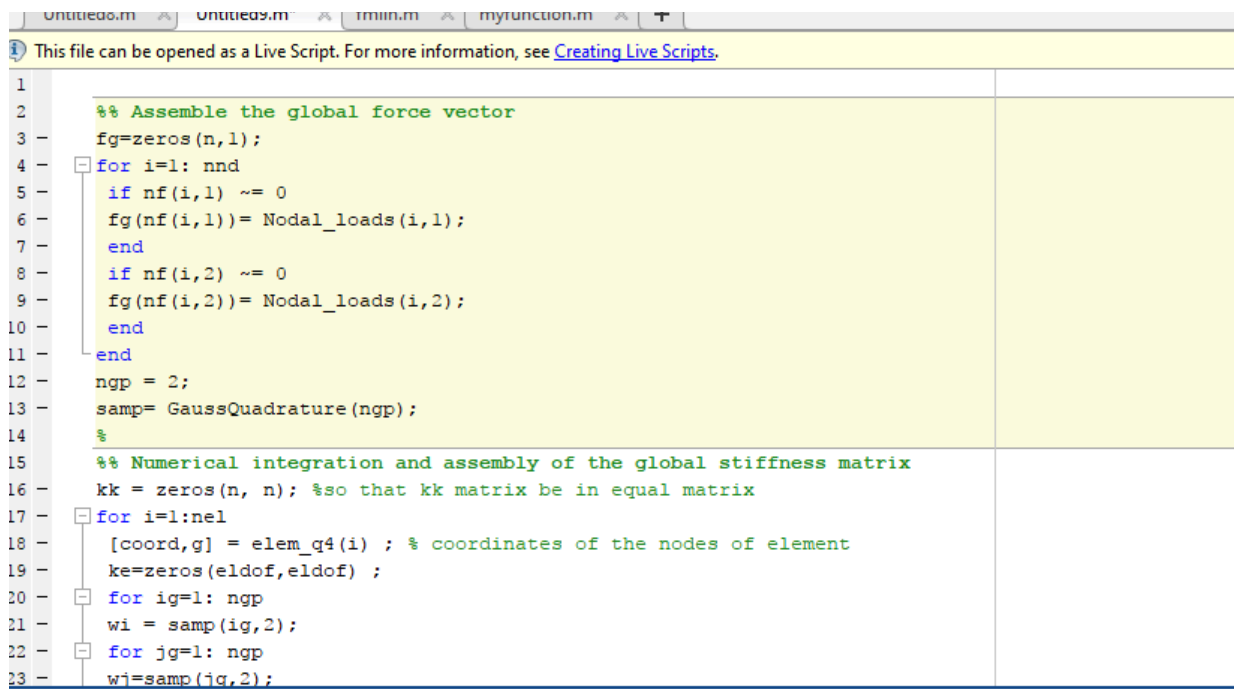
Figure 3 Element Stiffness matrix symbolical

Global Matrix

A screenshot of a MATLAB script editor window. The window has several tabs at the top: 'Untitled8.m', 'Untitled9.m', 'fmlin.m', and 'myfunction.m'. The active tab is 'myfunction.m'. A yellow banner at the top of the editor says 'This file can be opened as a Live Script. For more information, see [Creating Live Scripts](#).' The script code is as follows:

```
16 %  
17 %% Numerical integration and assembly of the global stiffness matrix  
18 kk = zeros(n, n);  
19 for i=1:nel  
20     [coord,g] = elem_q4(i) ; % coordinates of the nodes of element  
21     ke=zeros(eldof,eldof) ;  
22     for ig=1:ngp  
23         wi = samp(ig,2);  
24         for jg=1:ngp  
25             wj=samp(jg,2);  
26             [der,fun] = fmlin(samp, ig,jg); % Derivative of shape functions  
27             jac=der*coord; % Compute Jacobian matrix  
28             d=det(jac); % Compute determinant of Jacobian  
29             jac1=inv(jac); % Compute inverse of the Jacobian  
30             deriv=jac1*der; % Derivative of shape functions  
31             bee=formbee(deriv,nne,eldof); % Form matrix [B]  
32             ke=ke + d*thick*wi*wj*bee'*dee*bee; % Integrate stiffness matrix  
33         end  
34     end  
35     kk=form_kk(kk,ke, g); % assemble global stiffness matrix  
36 end  
37 %% solve for unknown displacements  
38 delta = kk\fg;
```

Figure 4 Global Matrix code

A screenshot of a MATLAB script editor window. The window has several tabs at the top: 'Untitled8.m', 'Untitled9.m', 'fmlin.m', and 'myfunction.m'. The active tab is 'myfunction.m'. A yellow banner at the top of the editor says 'This file can be opened as a Live Script. For more information, see [Creating Live Scripts](#).' The script code is as follows:

```
1  
2 %% Assemble the global force vector  
3 fg=zeros(n,1);  
4 for i=1:nnd  
5     if nf(i,1) ~= 0  
6         fg(nf(i,1))= Nodal_loads(i,1);  
7     end  
8     if nf(i,2) ~= 0  
9         fg(nf(i,2))= Nodal_loads(i,2);  
10    end  
11 end  
12 ngp = 2;  
13 samp= GaussQuadrature(ngp);  
14 %  
15 %% Numerical integration and assembly of the global stiffness matrix  
16 kk = zeros(n, n); %so that kk matrix be in equal matrix  
17 for i=1:nel  
18     [coord,g] = elem_q4(i) ; % coordinates of the nodes of element  
19     ke=zeros(eldof,eldof) ;  
20     for ig=1:ngp  
21         wi = samp(ig,2);  
22         for jg=1:ngp  
23             wj=samp(jg,2);
```

Figure 5 Global force vector

Results

1. MATLAB

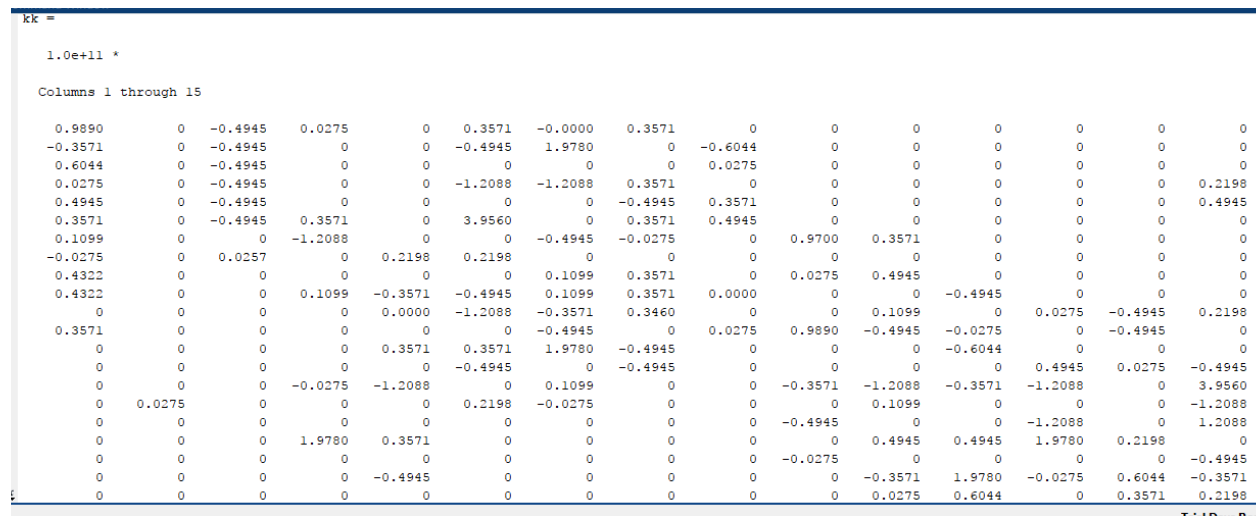


Figure 6 Global matrix result 1

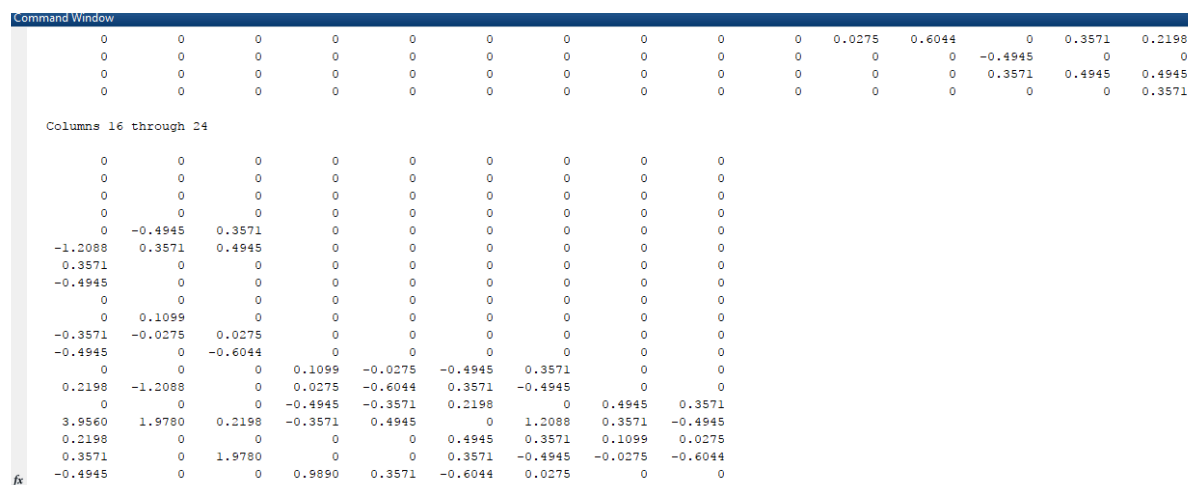


Figure 7 Global matrix result 2

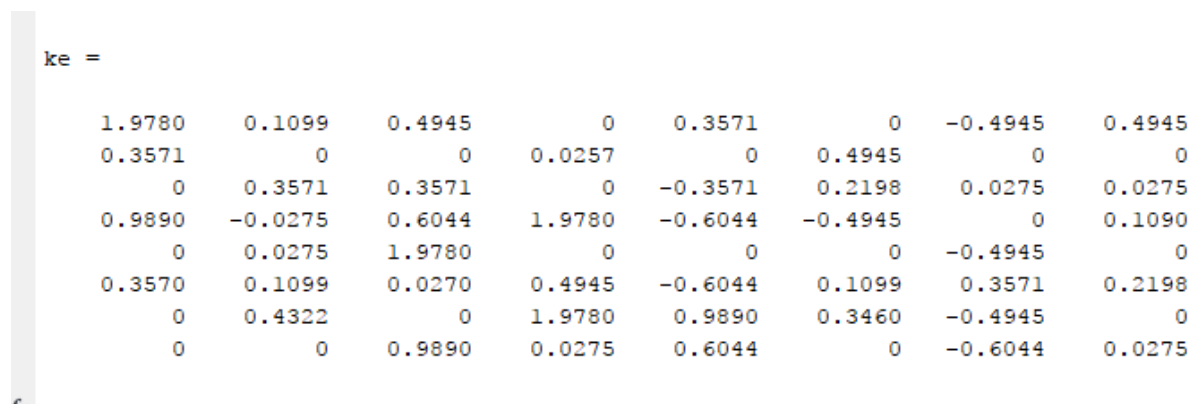


Figure 8 Reduced Matrix result

2. Hand Calculation

Plane Problems

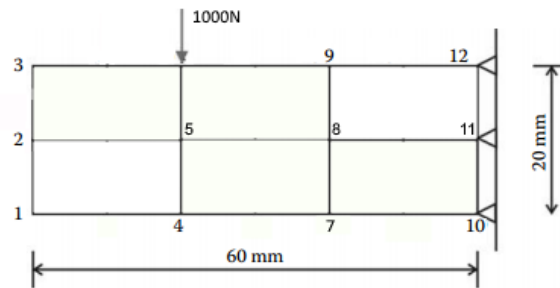


FIGURE 9.49 Finite element discretization with 4-noded quadrilateral elements.

Figure 9: Sample problem bi-linear quadrilateral (Khennane, 2013)

The problem sourced from a book above is a rectangular 2-D plane stress element. Given in the problem description where the dimensions of model, boundary conditions and relevant material properties (Young's modulus, Poisson ratio etc.). It is divided into 12 elements with 4 nodes/element.

Solution

Displacement assumptions:

$$u = \alpha_1 + \alpha_2 x + \alpha_3 y + \alpha_4 xy$$

$$v = \alpha_5 + \alpha_6 x + \alpha_7 y + \alpha_8 xy$$

Or in matrix form:

$$\begin{Bmatrix} u \\ v \end{Bmatrix} = \begin{bmatrix} 1 & x & y & xy & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & x & y & xy \end{bmatrix} \begin{Bmatrix} \alpha_1 \\ \alpha_2 \\ \alpha_3 \\ \alpha_4 \\ \alpha_5 \\ \alpha_6 \\ \alpha_7 \\ \alpha_8 \end{Bmatrix} \rightarrow \{u\} = [\Psi]\{\alpha\}$$

(1)

Where $[\Psi]$ an approximation is function matrix, and $\{\alpha\}$ is representing the “coefficients” or “generalized coordinates. However, the vector $\{\alpha\}$ can be obtained in term of the nodal displacement as following:

$$u_i = \alpha_1 + \alpha_2 x_i + \alpha_3 y_i + \alpha_4 x_i y_i$$

$$u_j = \alpha_1 + \alpha_2 x_j + \alpha_3 y_j + \alpha_4 x_j y_j$$

$$u_k = \alpha_1 + \alpha_2 x_k + \alpha_3 y_k + \alpha_4 x_k y_k$$

$$u_m = \alpha_1 + \alpha_2 x_m + \alpha_3 y_m + \alpha_4 x_m y_m$$

$$v_m = \alpha_5 + \alpha_6 x_m + \alpha_7 y_m + \alpha_8 x_m y_m$$

$$v_i = \alpha_5 + \alpha_6 x_i + \alpha_7 y_i + \alpha_8 x_i y_i$$

$$v_j = \alpha_5 + \alpha_6 x_j + \alpha_7 y_j + \alpha_8 x_j y_j$$

$$v_k = \alpha_5 + \alpha_6 x_k + \alpha_7 y_k + \alpha_8 x_k y_k$$

Written in matrix form

$$\begin{Bmatrix} u_i \\ u_j \\ u_k \\ u_m \end{Bmatrix} = \begin{bmatrix} 1 & x & y & xy \\ 1 & x & y & xy \\ 1 & x & y & xy \\ 1 & x & y & xy \end{bmatrix} \begin{Bmatrix} \alpha_1 \\ \alpha_2 \\ \alpha_3 \\ \alpha_4 \end{Bmatrix} \quad \& \quad \begin{Bmatrix} v_i \\ v_j \\ v_k \\ v_m \end{Bmatrix} = \begin{bmatrix} 1 & x & y & xy \\ 1 & x & y & xy \\ 1 & x & y & xy \\ 1 & x & y & xy \end{bmatrix} \begin{Bmatrix} \alpha_5 \\ \alpha_6 \\ \alpha_7 \\ \alpha_8 \end{Bmatrix}$$

Combining the two matrices:

$$\begin{Bmatrix} u_i \\ v_j \\ u_j \\ v_j \\ u_k \\ v_k \\ u_m \\ v_m \end{Bmatrix} = \begin{bmatrix} 1 & x & y & xy & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & x & y & xy \\ 1 & x & y & xy & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & x & y & xy \\ 1 & x & y & xy & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & x & y & xy \\ 1 & x & y & xy & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & x & y & xy \end{bmatrix} \begin{Bmatrix} \alpha_1 \\ \alpha_2 \\ \alpha_3 \\ \alpha_4 \\ \alpha_5 \\ \alpha_6 \\ \alpha_7 \\ \alpha_8 \end{Bmatrix}$$

$$\begin{Bmatrix} \bar{u} \end{Bmatrix} = [A] \{\alpha\} \quad \rightarrow \quad \{\alpha\} = [A]^{-1} \begin{Bmatrix} \bar{u} \end{Bmatrix} \quad (2)$$

Substituting equation (2) into (1):

$$\{u\} = [\Psi][A]^{-1} \begin{Bmatrix} \bar{u} \end{Bmatrix}$$

$$u = \frac{1}{2\Delta} \begin{bmatrix} (a_i + b_i x + c_i y + d_i xy) & (a_j + b_j x + c_j y + d_j xy) & (a_k + b_k x + c_k y + d_k xy) & (a_m + b_m x + c_m y + d_m xy) \end{bmatrix} \begin{Bmatrix} u_i \\ u_j \\ u_k \\ u_m \end{Bmatrix}$$

$$v = \frac{1}{2\Delta} \left[\begin{matrix} (a_i + b_i x + c_i y + d_i xy) & (a_j + b_j x + c_j y + d_j xy) & (a_k + b_k x + c_k y + d_k xy) & (a_m + b_m x + c_m y + d_m xy) \end{matrix} \right] \begin{Bmatrix} v_i \\ v_j \\ v_k \\ v_m \end{Bmatrix}$$

$$= \begin{bmatrix} 1 & x & y & xy \\ 1 & x & y & xy \\ 1 & x & y & xy \\ 1 & x & y & xy \end{bmatrix}$$

Where 2Δ = 2*(area of the rectangular) & a_i, b_i, c_i and d_i are:

$$a_i = b_i = c_i = x_k - x_j - xy$$

$$c_i = x_m - x_m - xy$$

Respectively follow the cycle permutation to obtain other equations for J and K and m. So:

$$\begin{Bmatrix} u \\ v \end{Bmatrix} = \frac{1}{2\Delta} \begin{bmatrix} (a_i + b_i x + c_i y + d_i xy) & 0 & (a_i + b_i x + c_i y + d_i xy) & 0 & (a_i + b_i x + c_i y + d_i xy) & 0 & (a_i + b_i x + c_i y + d_i xy) & 0 \\ 0 & (a_i + b_i x + c_i y + d_i xy) & 0 & (a_i + b_i x + c_i y + d_i xy) & 0 & (a_i + b_i x + c_i y + d_i xy) & 0 & (a_i + b_i x + c_i y + d_i xy) \end{bmatrix} \begin{Bmatrix} u \\ v \end{Bmatrix}$$

$$\{u\} = [N]\{\bar{u}\}$$

- The [N] matrix comes out to be

$$[N] = \begin{bmatrix} N_1 & 0 & N_2 & 0 & N_3 & 0 & N_4 & 0 \\ 0 & N_1 & 0 & N_2 & 0 & N_3 & 0 & N_4 \end{bmatrix},$$

$$N_1 = \frac{1}{4bc} (x_e y_e + c x_e + b y_e + bc) - \frac{x}{4bc} (y_e + c) - \frac{y}{4bc} (x_e + b) + \frac{xy}{4bc}$$

$$N_2 = \frac{1}{4bc} (-x_e y_e - c x_e + b y_e + bc) + \frac{x}{4bc} (y_e + c) - \frac{y}{4bc} (-x_e + b) - \frac{xy}{4bc}$$

$$N_3 = \frac{1}{4bc} (x_e y_e - c x_e - b y_e + bc) + \frac{x}{4bc} (-y_e + c) + \frac{y}{4bc} (-x_e + b) + \frac{xy}{4bc}$$

$$N_4 = \frac{1}{4bc} (-x_e y_e + c x_e - b y_e + bc) - \frac{x}{4bc} (-y_e + c) + \frac{y}{4bc} (x_e + b) - \frac{xy}{4bc}$$

$$\{\varepsilon\} = \frac{1}{2\Delta} \begin{bmatrix} b_i & 0 & b_j & 0 & b_k & 0 & b_m & 0 \\ 0 & c_i & 0 & c_j & 0 & c_k & 0 & c_m \\ c_i & b_i & c_j & b_j & c_k & b_k & c_m & b_m \end{bmatrix} \begin{Bmatrix} u_i \\ v_i \\ u_j \\ v_j \\ u_k \\ v_k \\ u_m \\ v_m \end{Bmatrix}$$

$$\{\varepsilon\} = [B] \{\bar{u}\}$$

where

$$[B] = \frac{1}{2\Delta} \begin{bmatrix} b_i & 0 & b_j & 0 & b_k & 0 & b_m & 0 \\ 0 & c_i & 0 & c_j & 0 & c_k & 0 & c_m \\ c_i & b_i & c_j & b_j & c_k & b_k & c_m & b_m \end{bmatrix}$$

So, to find the global stiffness matrix:

$$[K] = [B]^T [D] [B] (t, \Delta)$$

$$[k] = t \cdot \begin{bmatrix} \frac{D_{11}c + D_{33}b}{3b} + \frac{D_{12} + D_{33}}{4} & -\frac{D_{11}c + D_{33}b}{3b} + \frac{D_{12} + D_{33}}{4} & \frac{D_{12} - D_{33}}{4} & -\frac{D_{11}c - D_{33}b}{6b} - \frac{D_{12} - D_{33}}{4} & -\frac{D_{11}c - D_{33}b}{6b} - \frac{D_{12} - D_{33}}{4} & \frac{D_{11}c - D_{33}b}{6b} - \frac{D_{12} - D_{33}}{4} & \frac{D_{11}c - D_{33}b}{6b} - \frac{D_{12} - D_{33}}{4} & -\frac{D_{12} + D_{33}}{4} \\ \frac{D_{22}b + D_{33}c}{3c} + \frac{D_{12} + D_{33}}{4} & -\frac{D_{22}b + D_{33}c}{3c} + \frac{D_{12} + D_{33}}{4} & \frac{D_{22}b - D_{33}c}{6c} - \frac{D_{12} - D_{33}}{4} & -\frac{D_{22}b - D_{33}c}{6c} - \frac{D_{12} - D_{33}}{4} & -\frac{D_{22}b - D_{33}c}{6c} - \frac{D_{12} - D_{33}}{4} & \frac{D_{22}b - D_{33}c}{6c} - \frac{D_{12} - D_{33}}{4} & \frac{D_{22}b - D_{33}c}{6c} - \frac{D_{12} - D_{33}}{4} & -\frac{D_{22}b + D_{33}c}{3c} + \frac{D_{12} + D_{33}}{4} \\ \frac{D_{11}c + D_{33}b}{3b} + \frac{D_{12} + D_{33}}{4} & -\frac{D_{11}c + D_{33}b}{3b} + \frac{D_{12} + D_{33}}{4} & \frac{D_{12} - D_{33}}{4} & -\frac{D_{11}c - D_{33}b}{6b} - \frac{D_{12} - D_{33}}{4} & -\frac{D_{11}c - D_{33}b}{6b} - \frac{D_{12} - D_{33}}{4} & \frac{D_{11}c - D_{33}b}{6b} - \frac{D_{12} - D_{33}}{4} & \frac{D_{11}c - D_{33}b}{6b} - \frac{D_{12} - D_{33}}{4} & -\frac{D_{12} + D_{33}}{4} \\ \frac{D_{22}b + D_{33}c}{3c} + \frac{D_{12} + D_{33}}{4} & -\frac{D_{22}b + D_{33}c}{3c} + \frac{D_{12} + D_{33}}{4} & \frac{D_{22}b - D_{33}c}{6c} - \frac{D_{12} - D_{33}}{4} & -\frac{D_{22}b - D_{33}c}{6c} - \frac{D_{12} - D_{33}}{4} & -\frac{D_{22}b - D_{33}c}{6c} - \frac{D_{12} - D_{33}}{4} & \frac{D_{22}b - D_{33}c}{6c} - \frac{D_{12} - D_{33}}{4} & \frac{D_{22}b - D_{33}c}{6c} - \frac{D_{12} - D_{33}}{4} & -\frac{D_{22}b + D_{33}c}{3c} + \frac{D_{12} + D_{33}}{4} \\ \frac{D_{11}c + D_{33}b}{3b} + \frac{D_{12} + D_{33}}{4} & -\frac{D_{11}c + D_{33}b}{3b} + \frac{D_{12} + D_{33}}{4} & \frac{D_{12} - D_{33}}{4} & -\frac{D_{11}c - D_{33}b}{6b} - \frac{D_{12} - D_{33}}{4} & -\frac{D_{11}c - D_{33}b}{6b} - \frac{D_{12} - D_{33}}{4} & \frac{D_{11}c - D_{33}b}{6b} - \frac{D_{12} - D_{33}}{4} & \frac{D_{11}c - D_{33}b}{6b} - \frac{D_{12} - D_{33}}{4} & -\frac{D_{12} + D_{33}}{4} \\ \frac{D_{22}b + D_{33}c}{3c} + \frac{D_{12} + D_{33}}{4} & -\frac{D_{22}b + D_{33}c}{3c} + \frac{D_{12} + D_{33}}{4} & \frac{D_{22}b - D_{33}c}{6c} - \frac{D_{12} - D_{33}}{4} & -\frac{D_{22}b - D_{33}c}{6c} - \frac{D_{12} - D_{33}}{4} & -\frac{D_{22}b - D_{33}c}{6c} - \frac{D_{12} - D_{33}}{4} & \frac{D_{22}b - D_{33}c}{6c} - \frac{D_{12} - D_{33}}{4} & \frac{D_{22}b - D_{33}c}{6c} - \frac{D_{12} - D_{33}}{4} & -\frac{D_{22}b + D_{33}c}{3c} + \frac{D_{12} + D_{33}}{4} \\ \frac{D_{11}c + D_{33}b}{3b} + \frac{D_{12} + D_{33}}{4} & -\frac{D_{11}c + D_{33}b}{3b} + \frac{D_{12} + D_{33}}{4} & \frac{D_{12} - D_{33}}{4} & -\frac{D_{11}c - D_{33}b}{6b} - \frac{D_{12} - D_{33}}{4} & -\frac{D_{11}c - D_{33}b}{6b} - \frac{D_{12} - D_{33}}{4} & \frac{D_{11}c - D_{33}b}{6b} - \frac{D_{12} - D_{33}}{4} & \frac{D_{11}c - D_{33}b}{6b} - \frac{D_{12} - D_{33}}{4} & -\frac{D_{12} + D_{33}}{4} \\ \frac{D_{22}b + D_{33}c}{3c} + \frac{D_{12} + D_{33}}{4} & -\frac{D_{22}b + D_{33}c}{3c} + \frac{D_{12} + D_{33}}{4} & \frac{D_{22}b - D_{33}c}{6c} - \frac{D_{12} - D_{33}}{4} & -\frac{D_{22}b - D_{33}c}{6c} - \frac{D_{12} - D_{33}}{4} & -\frac{D_{22}b - D_{33}c}{6c} - \frac{D_{12} - D_{33}}{4} & \frac{D_{22}b - D_{33}c}{6c} - \frac{D_{12} - D_{33}}{4} & \frac{D_{22}b - D_{33}c}{6c} - \frac{D_{12} - D_{33}}{4} & -\frac{D_{22}b + D_{33}c}{3c} + \frac{D_{12} + D_{33}}{4} \end{bmatrix}$$

Symm

Where D_{11}, D_{22}, D_{33} and D_{44} and so on are the components of the constitutive stress strain matrix [D].

The [D] matrix for plain stress can be calculated as

Now applying the above matrix for each element, we get the global matrix

$$[K] = \begin{bmatrix} 19.67 & 9.77 & 49.42 & 0 & 35.71 & 0 & -49.42 & -48.52 \\ 34.75 & 0 & 0 & 2.57 & 0 & 49.42 & 0 & 0 \\ 0 & 34.71 & 35.71 & 0 & -34.71 & 21.98 & 2.75 & 72.11 \\ 98.90 & -2.75 & 59.75 & 197.75 & -59.75 & 49.42 & 0 & -28.85 \\ 0 & 2.75 & 2.75 & 0 & 0 & 0 & -49.42 & 0 \\ 34.75 & 9.77 & -2.75 & 48.52 & 59.75 & 9.77 & 35.71 & 9.61 \\ 0 & 43.22 & 0 & 197.5 & 98.23 & 34.60 & 49.35 & 0 \\ 0 & 0 & 98.90 & 2.75 & 59.75 & 0 & 59.75 & 28.85 \end{bmatrix} \times 10^9$$

- From the element matrices we can find the global stiffness matrix for the problem.

The unreduced global matrix size comes out to be $[8] \times [8]$ matrix.

3. ANSYS

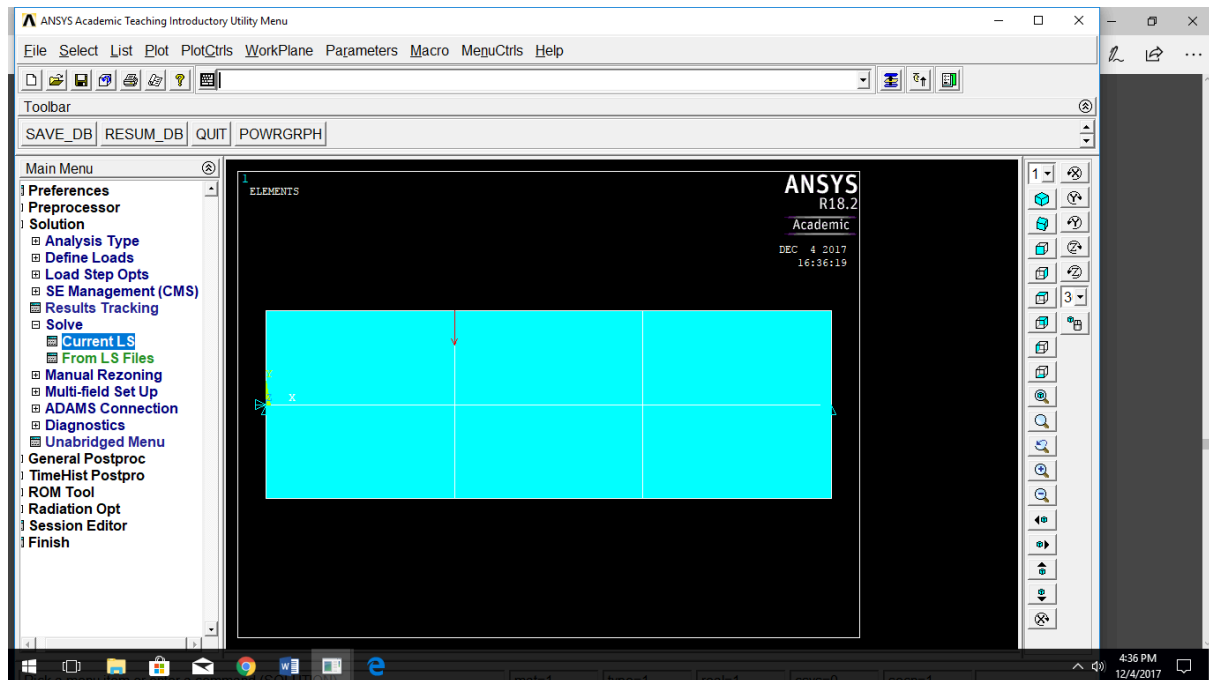


Figure 10 Ansys Model

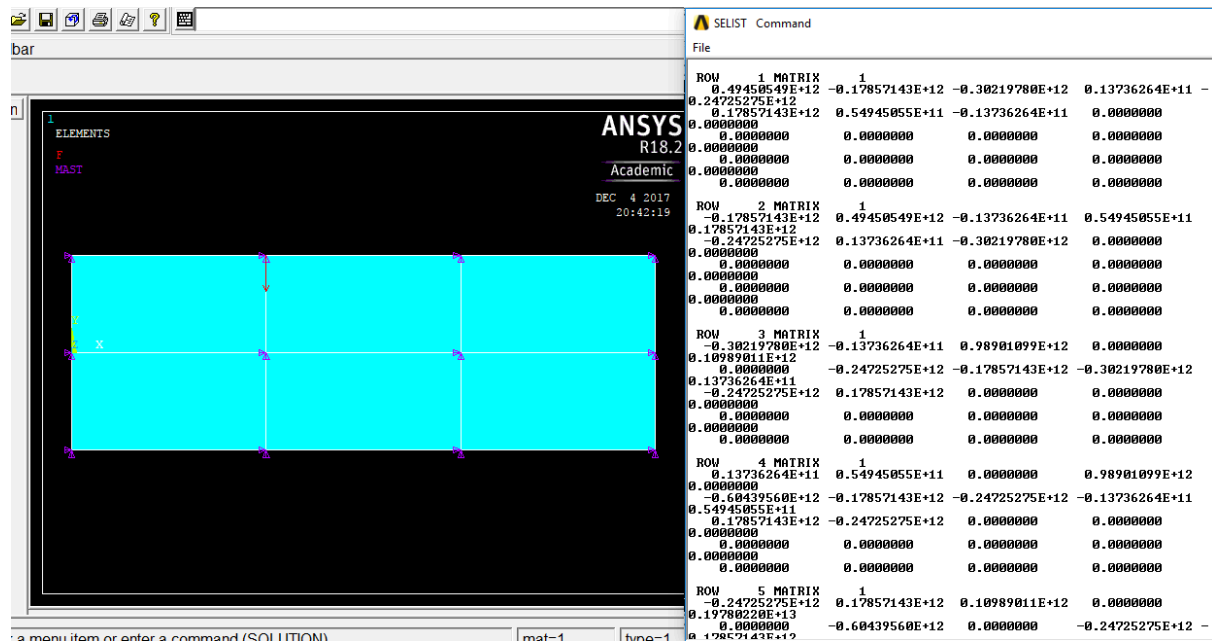


Figure 11 Stiffness matrix from Ansys

Verifications of results

The most important step of this project was to verify results across ANSYS, MATLAB and hand-calculations. The results that are obtained from Ansys and the MATLAB code we devised were not identical. Our element was designed to be 12 nodes and with 2 Degrees of Freedom (DOF) per node, a 24 x 24 global matrix was to be formulated. The percentage error between the hand calculation and the results from MATLAB was calculated to be $\frac{35.75-34.75}{35.75} \times 100 = 2.8\%$. The low percentage difference is because the number of elements that was used in hand calculation is same as the results obtained from MATLAB. The result that we got from MATLAB was the same as the one found using hand calculations since we used the same problem and divided the element into six elements instead of one which shows that for future references MATLAB should be used. The forces and the problem were identical. Therefore, the results obtained has almost negligible percentage error. However, in the case of Ansys and MATLAB calculations we found that $\frac{34.75-30.78}{30.78} \times 100 = 12.9\%$. This could be the cause of many reasons such as:

1. The stacking of the elements.
2. Does not take account of factors such as displacement values or meshing.
3. Matlab has a certain limit to handle the matrix values.

Conclusion

To sum up this final mini-project, a code was developed to find the element nodal shape functions, element stiffness matrix, global matrix of a plane-stress quadrilateral. For this task, commercial software's MATLAB and ANSYS were employed to derive the global unreduced stiffness matrix. The element type in ANSYS can be defined but it's limited to a certain shape which restricts the user to generalize it even in the case given above. However, it is noted that using ANSYS is time consuming since to calculate the stiffness matrix the user must draw the element by a fairly timely process (Key-points, Lines, Areas, Meshing, Constraints). Finally, the global stiffness matrix was found out.

Bibliography

Gadala, M. S. (2017). *Final Mini Project*.

Khennane, A. (2013). *Introduction to Finite Element Analysis Using MATLAB® and Abaqus*. CRC Press.

MATLAB. (2017). *MATLAB Documentation*. Retrieved from MATLAB:
<https://www.mathworks.com/help/matlab/>

Appendix

```
nnd = 12; % Number of nodes:
nel = 6; % Number of elements:
nne = 4; % Number of nodes per element:
nodof = 2; % Number of degrees of freedom per node
ngp = 2; % number of Gauss points
eldof = nne*nodof; % Number of degrees of freedom per element
%% Nodes coordinates x and y, this loop generates coordinates
geom = zeros(nnd,2);
xcr = 0:0.15:1.2;
ycr = 0:0.15:0.3;
for i = 1:size(xcr,2)
    v1 = zeros(size(ycr,2),2); % General vector of 5 by 2
    for j = 1:size(ycr,2)
        coord = [xcr(i) ycr(j)];
        v1(j,:) = v1(j,)+coord;
    end
    geom(((3*i-2):3*i),:) = geom(((3*i-2):3*i),:)+v1;
end
%
%% Element connectivity
connec = [ 1 4 5 2; 2 5 6 3; 4 7 8 5; 5 8 9 6; 7 10 11 8; 8 11 12 9];...

%
%% Material
E = 200*10^6; % Elastic modulus in kN/m2
nu = 0.25; % Poisson's ratio
thick = 0.025; % Beam thickness in m
%
%% Form the elastic matrix for plane stress
F2 = E/(1-nu^2);
F3 = 0.5*(1-nu);
dee= [F2 F2*nu 0 ; F2*nu F2 0; 0 0 F2*F3];
%% Boundary conditions
nf = ones(nnd, nodof); % Initialize the matrix nf to 1
nf(1,:) = [0 0]; % Node 19 is restrained in the x and y directions
nf(2,:) = [0 0]; % Node 20 is restrained in the x and y directions
nf(3,:) = [0 0]; % Node 21 is restrained in the x and y directions
%
%% Counting of the free degrees of freedom
n=0;
for i=1:nnd
    for j=1:nodof
        if nf(i,j) ~= 0
            n=n+1;
        end
    end
end
```



```

    nf(i,j)=n;
end
end
end
%% Nodal Loading
Nodal_loads= zeros(nnd, 2); % Initialize the matrix of nodal loads to 0
%% Apply a concentrated at the node having x = 0, and y = 0.
Nodal_loads(25,:) = [0. -33.75];
Nodal_loads(26,:) = [0. -112.5];
Nodal_loads(27,:) = [0. -33.75];
%

function[der,fun] = fmlin(samp, ig,jg)
%
% This function returns the vector of the shape function and their
% derivatives with respect to xi and eta
%
xi=samp(ig,1);
eta=samp(jg,1);
%
fun = 0.25*[(1.- xi - eta + xi*eta);...
(1.+ xi - eta - xi*eta);...
(1.+ xi + eta + xi*eta);...
(1.- xi+ eta - xi*eta)];
%
der = 0.25*[-(1-eta) (1-eta) (1+eta) -(1+eta);...
-(1-xi) -(1+xi) (1+xi) (1-xi)];
end

%% Assemble the global force vector
fg=zeros(n,1);
for i=1: nnd
    if nf(i,1) ~= 0
        fg(nf(i,1))= Nodal_loads(i,1);
    end
    if nf(i,2) ~= 0
        fg(nf(i,2))= Nodal_loads(i,2);
    end
end
ngp = 2;
samp= GaussQuadrature(ngp);
%
%% Numerical integration and assembly of the global stiffness matrix
kk = zeros(n, n); %so that kk matrix be in equal matrix
for i=1:nel
    [coord,g] = elem_q4(i) ; % coordinates of the nodes of element
    ke=zeros(eldof,eldof) ;
    for ig=1: ngp
        wi = samp(ig,2);
        for jg=1: ngp
            wj=samp(jg,2);
            [der,fun] = fmlin(samp, ig,jg); % Derivative of shape functions
            jac=der*coord; % Compute Jacobian matrix
            d=det(jac); % Compute determinant of Jacobian
            jac1=inv(jac); % Compute inverse of the Jacobian
            deriv=jac1*der; % Derivative of shape functions
            bee=formbee(deriv,nne,eldof); % Form matrix [B]
            ke=ke + d*thick*wi*wj*bee'*dee*bee; % Integrate stiffness matrix
        end
    end
end
end

```

```

    kk=form_kk(kk,ke, g); % assemble global stiffness matrix
end
%% solve for unknown displacements
delta = kk\fg;
for i=1: nnd %
    if nf(i,1) == 0
        x_disp =0;
    else
        x_disp = delta(nf(i,1));
    end
    if nf(i,2) == 0
        y_disp = 0;
    else
        y_disp = delta(nf(i,2));
    end
    disp([i x_disp y_disp]) % Display displacements of each node
    DISP(i,:) = [x_disp y_disp];
end
%% Calculate stresses and strains at the center of each element
ngp=1;
samp=gauss(ngp);
for i=1:nel
    [coord,g] = elem_q4(i); % coordinates of the nodes of element
    eld=zeros(eldof,1);
    for m = 1:eldof
        if g(m) == 0
            eld(m) = 0;
        else
            eld(m)=delta(g(m));
        end
    end
    for ig = 1: ngp
        wi = samp(ig,2);
        for jg = 1: ngp
            wj = samp(jg,2);
            [der,fun] = fmlin(samp, ig,jg); % Derivative of shape functions
            jac = der*coord; % Compute Jacobian matrix
            jac1 = inv(jac); % Compute inverse of the Jacobian
            deriv = jac1*der; % Derivative of shape functions
            bee = formbee(deriv,nne,eldof); % Form matrix [B]
            eps = bee*eld; % Compute strains
            sigma = dee*eps; % Compute stresses
        end
    end
    SIGMA(i,:) = sigma; % Store stresses for all elements
end
%% Average stresses at nodes
[Zx, Zy, Zt, Z1, Z2]=stresses_at_nodes_Q4(SIGMA);
%% Plot stresses in the x_direction
U2 = DISP(:,2);
cmin = min(U2);
cmax = max(U2);
caxis([cmin cmax]);

```